

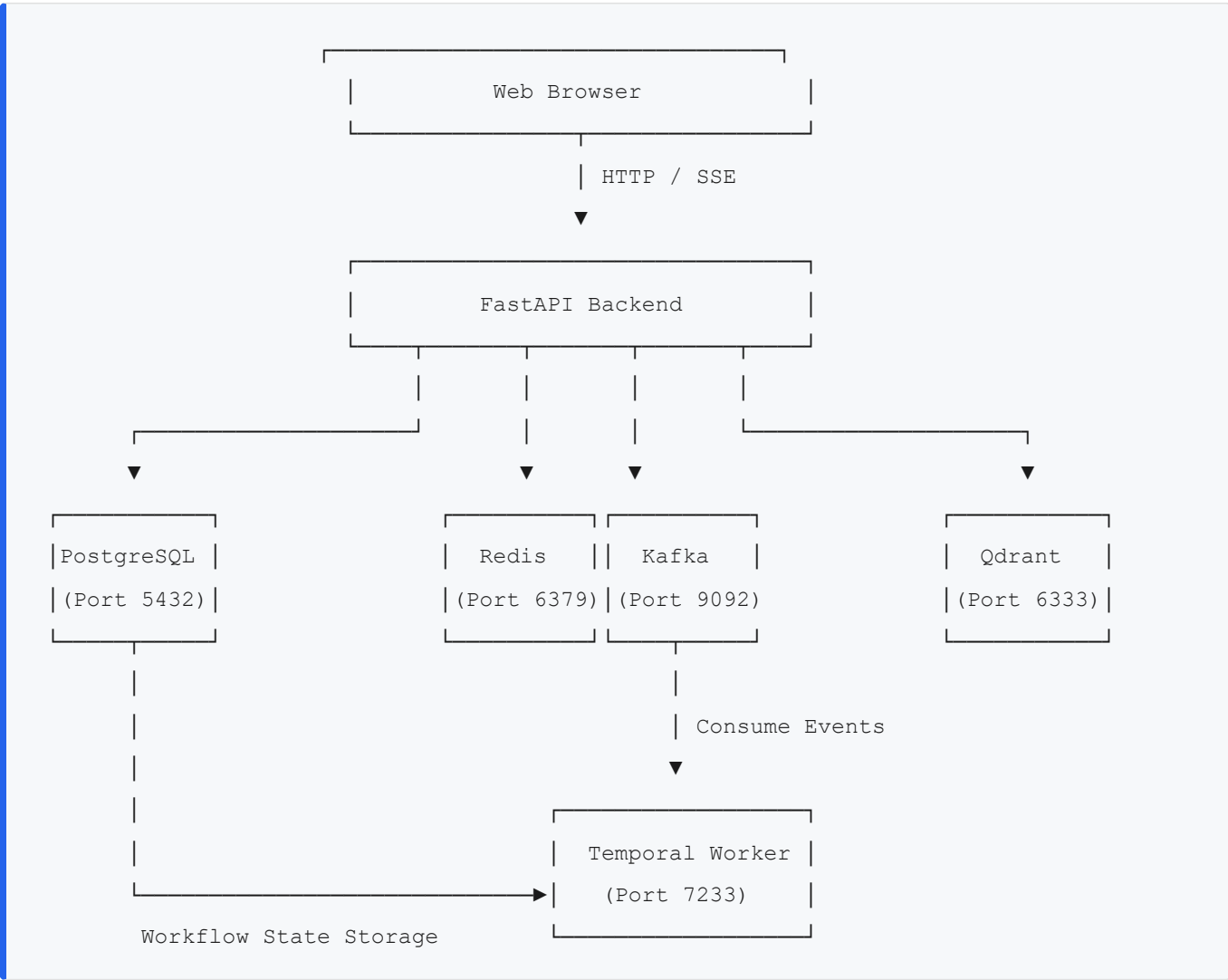
PROJECT 31A — PLATFORM INFRASTRUCTURE BIBLE (DOCUMENT 7)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

1. INFRASTRUCTURE TOPOLOGY OVERVIEW

Project 31A is built on a microservices infrastructure designed to support high-throughput Web scraping, AI model execution, durable workflow orchestration, event-driven streaming, and isolated multi-tenant data storage.



The infrastructure runs inside a containerized setup defined by `docker-compose.yml` in local development and staging environments. Production deployments utilize the configurations declared under the `/infrastructure` directory, including Kubernetes manifests and Terraform definitions.

2. SERVICE REGISTRY (DEVELOPMENT & STAGING)

The development infrastructure runs nine containerized services. Each service is configured with `restart: unless-stopped` to ensure recoverability following host restarts.

2.1 Database Engine (`seo-postgres`)

- **Container Name:** `seo-postgres`
- **Image:** `postgres:16-alpine`
- **Ports:** `5432:5432`
- **Volume Mounts:** `postgres_data:/var/lib/postgresql/data`
- **Health Check:** `pg_isready -U postgres -d seo_platform` (runs every 10s, timeout 5s, retries 5)
- **Purpose:** Primary relational persistence. Stores tenants, users, clients, campaigns, prospects, outreach threads, keywords, recommendations, audit logs, and temporal system tables.

2.2 Memory State & Cache Layer (`seo-redis`)

- **Container Name:** `seo-redis`
- **Image:** `redis:7-alpine`
- **Ports:** `6379:6379`
- **Volume Mounts:** `redis_data:/data`
- **Configuration:** `redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy allkeys-lru`
- **Purpose:** Stores transient operational states (tokens, active locks), rate-limiting buckets, idempotency keys, SRE kill switch states, and SSE event dispatch pub/sub queues.

2.3 Workflow Orchestrator (`seo-temporal`)

- **Container Name:** `seo-temporal`
- **Image:** `temporalio/auto-setup:1.24`
- **Ports:** `7233:7233`
- **Volume Mounts:** `./temporal-config:/etc/temporal/config`
- **Environment:** Connected directly to `seo-postgres` (requires database name `temporal_db` and user `temporal`).
- **Purpose:** Temporal cluster gateway. Manages task queue registrations, schedules workflows, handles signal dispatches, and persists state histories.

2.4 Temporal UI Dashboard (`seo-temporal-ui`)

- **Container Name:** `seo-temporal-ui`
- **Image:** `temporalio/ui:2.26.2`
- **Ports:** `8233:8080` (maps local port 8233 to container 8080)
- **Environment:** `TEMPORAL_ADDRESS=temporal:7233`
- **Purpose:** Admin console for monitoring active workflows, replays, and signals.

2.5 Message Broker (`seo-kafka` & `seo-zookeeper`)

- **Container Names:** `seo-kafka`, `seo-zookeeper`
- **Images:** `confluentinc/cp-kafka:7.6.0`, `confluentinc/cp-zookeeper:7.6.0`
- **Ports:** ZooKeeper: `2181:2181`, Kafka: `9092:9092`
- **Volume Mounts:** `zookeeper_data`, `kafka_data`
- **Configuration:** Auto-create topics enabled; default message retention set to 7 days (`log.retention.hours=168`); max message size set to 1MB.
- **Purpose:** Decouples API mutations from heavy back-end worker actions via event streams.

2.6 Vector Database (`seo-qdrant`)

- **Container Name:** `seo-qdrant`
- **Image:** `qdrant/qdrant:v1.9.7`
- **Ports:** `6333:6333` (REST API), `6334:6334` (gRPC)
- **Volume Mounts:** `qdrant_data:/qdrant/storage`
- **Purpose:** Vector search engine. Stores keyword embeddings generated by `nv-embedqa-e5-v5` for semantic indexing.

2.7 Object Storage (`seo-minio`)

- **Container Name:** `seo-minio`
- **Image:** `minio/minio:latest`
- **Ports:** `9000:9000` (API), `9001:9001` (Admin Console)
- **Volume Mounts:** `minio_data:/data`
- **Purpose:** S3-compatible file storage. Stores compiled PDF reports, outreach email attachments, and HTML page scrapings.

2.8 Sandbox Email Inbox (`seo-mailhog`)

- **Container Name:** `seo-mailhog`
- **Image:** `mailhog/mailhog:latest`
- **Ports:** `1025:1025` (SMTP), `8025:8025` (Web UI console)
- **Purpose:** SMTP sandbox. Intercepts all outreach emails sent in development/staging.

3. NETWORKING & COMMUNICATIONS MATRIX

Inter-service communication is restricted to specific ports inside the Docker network (`seo_network`).

Source Service	Destination Service	Protocol	Port	Purpose
FastAPI Backend	seo-postgres	TCP	5432	Async database queries via <code>asyncpg</code> .
FastAPI Backend	seo-redis	TCP	6379	Rate-limiting, cache read/write, SSE.
FastAPI Backend	seo-temporal	gRPC	7233	Start workflows, trigger signals.
FastAPI Backend	seo-kafka	TCP	9092	Publish campaign lifecycle events.
Temporal Worker	seo-temporal	gRPC	7233	Long-poll task queues, submit results.
Temporal Worker	seo-postgres	TCP	5432	Persist prospect metrics, client records.
Temporal Worker	seo-redis	TCP	6379	Check kill switches, read/write locks.
Temporal Worker	seo-qdrant	gRPC	6334	Batch ingest keyword embeddings.
Temporal Worker	seo-minio	HTTP	9000	Upload compiled PDF reports.
Temporal Worker	seo-mailhog	SMTP	1025	Dispatch test outreach emails (dev only).

4. PERSISTENT STORAGE VOLUMES MAP

Data directories are mapped to named Docker volumes to prevent data loss on container rebuilds.

- `postgres_data` : Mounts `/var/lib/postgresql/data` . Contains the relational DB tables, indexes, and schema. (Critical for backup)
- `redis_data` : Mounts `/data` . Contains Redis AOF (Append-Only File) logs.
- `kafka_data` : Mounts `/var/lib/kafka/data` . Stores transient commit logs for event topics.
- `qdrant_data` : Mounts `/qdrant/storage` . Stores high-dimensional vector index files.
- `minio_data` : Mounts `/data` . Contains S3 bucket structures.
- `temporal_data` : Mounted inside PostgreSQL. Holds execution history tables.

5. DOCKER, KUBERNETES & TERRAFORM (PRODUCTION DEPLOYMENT)

Production deployment templates are organized under the `/infrastructure` directory to support scalable cloud platforms (GCP by default).

5.1 Terraform Provisioning (`/infrastructure/terraform/`)

Terraform configurations provision resources on Google Cloud Platform:

- `main.tf` / `db.tf` : Sets up Google Cloud SQL (PostgreSQL 16) with High Availability (HA) enabled.
- `gke.tf` : Provisions a Google Kubernetes Engine (GKE) autopilot cluster.
- `memorystore.tf` : Sets up Google Cloud Memorystore (Redis) for caching and rate limiting.
- `gcs.tf` : Provisions Cloud Storage buckets (replacing MinIO) for asset storage.

5.2 Kubernetes Orchestrator (`/infrastructure/kubernetes/`)

Applications are deployed as pods inside GKE:

- `api-deployment.yaml` : Deploys stateless FastAPI backend replicas behind an Ingress controller.
- `worker-deployment.yaml` : Deploys stateful Temporal workers. Scaled independently based on task queue depths.
- `hpa.yaml` : Horizontal Pod Autoscaler scaling worker pods based on CPU utilization and queue metrics.